

Google Stock Price Prediction Using RNN - LSTM Python

In this article, we are going to look at an outstanding end-to-end real-life Recurrent Neural Network (RNN) - LSTM project where we will predict the price of google stock. We will be using Python, Keras, Jupyter Notebook, and Tensorflow for this project.

Before directly diving into the code let's have an overview of the topics such as RNN and LSTM network.



STOCKS IMAGE

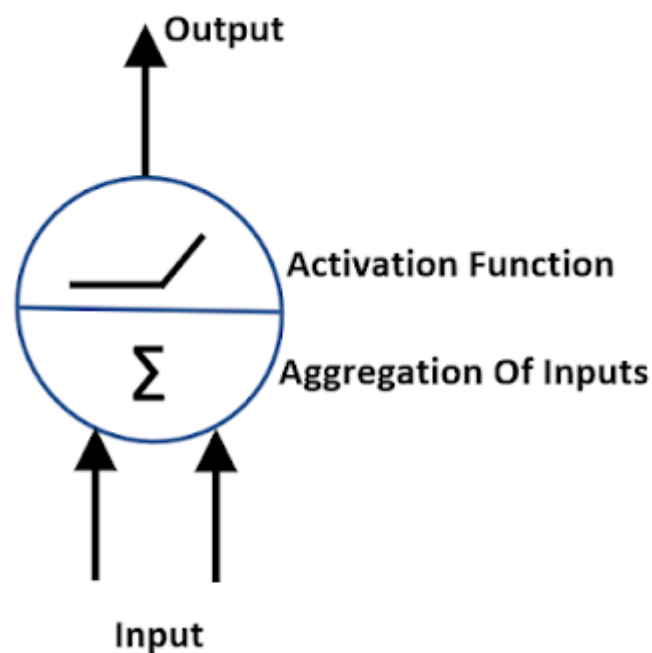
Q.)What is RNN?

RNN also is known as **Recurrent Neural Network** has unique properties that allow them to be more effective for sequence data, sequence data can be a variety of data sources it can be anything from timestamps sales data or sequence of text in a sentence or biological data like heartbeat data overtime etc.

Basically, RNN helps us to predict future sequential data more accurately which learns from history.

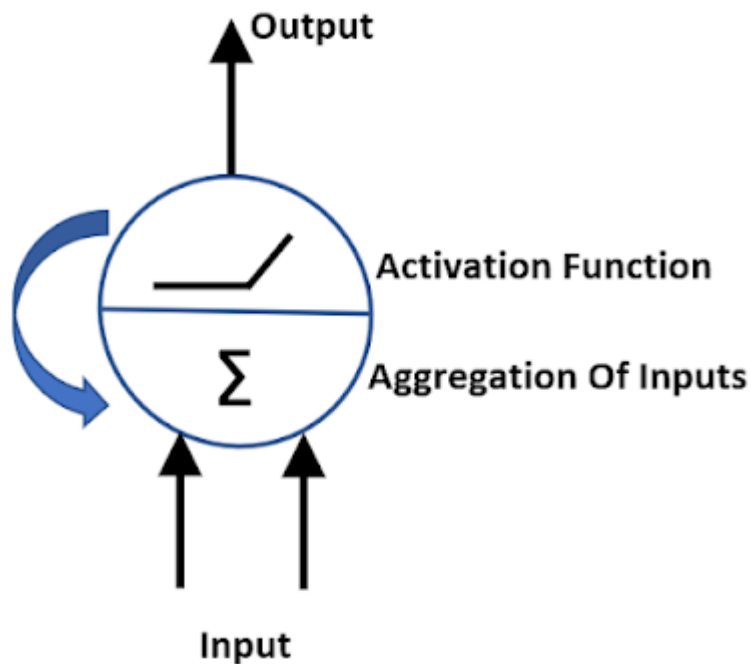
To do this properly, we need to somehow let the neuron “Know ” about its previous history of outputs. One of the easy ways to do this is to simply feed it’s output back into itself as an input.

Let’s look at the normal neuron in feed-forward network



NEURON

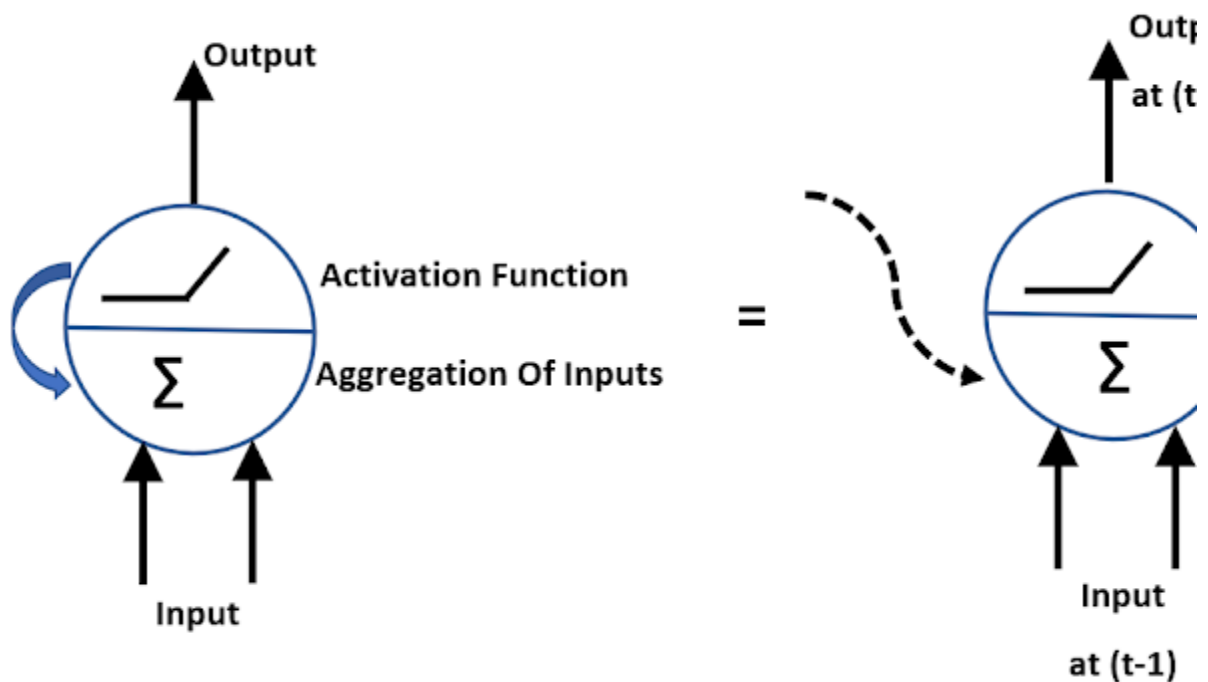
Now how we will take advantage of being able to relay back the history with a neuron?



NEURON INVERSE

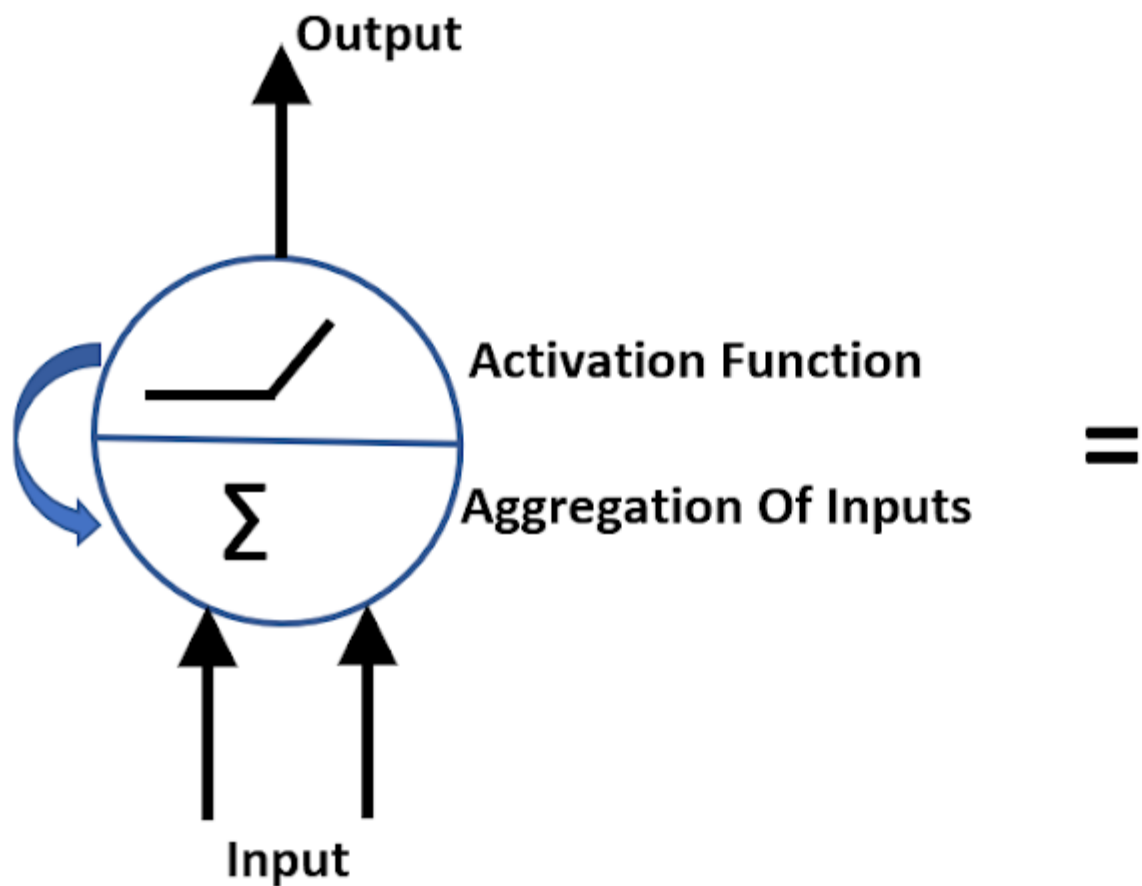
Here we can use a Recurrent Neuron. In Recurrent Neuron the main difference here not only sends the output out to the next layer, it takes that output and feeds it back to itself. So over time, we can unroll this.

Now go one timestamp into the past at Time($t-1$) so as you can see we have some input coming in at $t-1$ this can be batch of sequence data which are aggregated and passed to the activation function and we get the output of this Recurrent a neuron at $t-1$.



FUTURE NEURON

As time goes on we end up doing for that next input batch of another sequence or input at time t , here we are not only going to feed in the input at time t but we also feed in the information of the output of Recurrent neuron times $(t-1)$ which then give us the output at time t . Then we take that output at time t and feed it along with the input at a time $(t+1)$ that way we are retaining the historical information. This is the Recurrent neuron essentially unrolled throughout time.



Time

RECURRENT NEURON

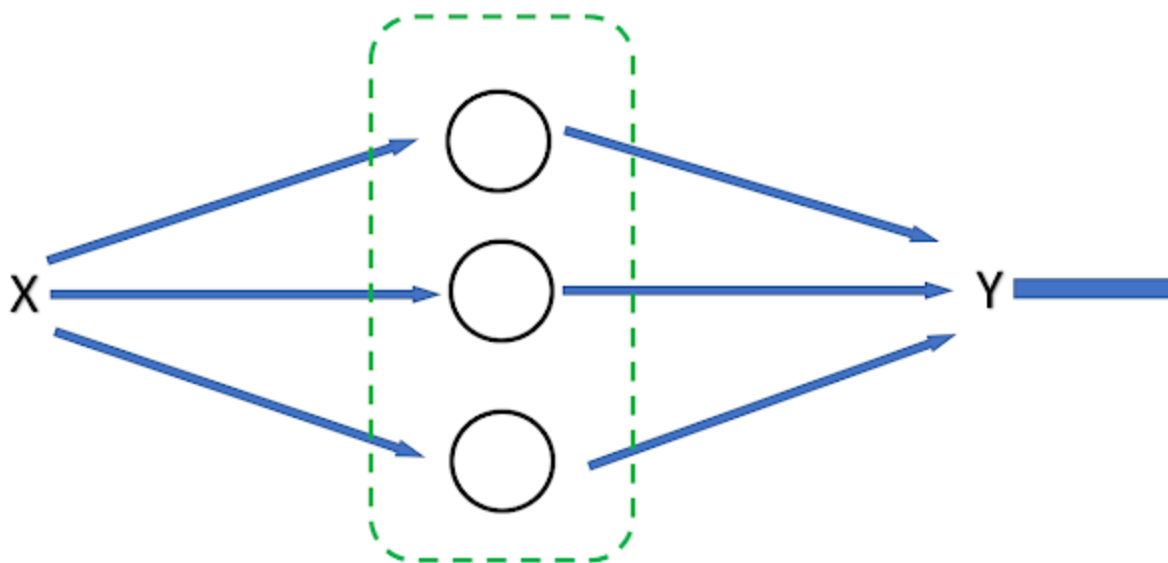
Cells that are a function of inputs from previous time steps are known as memory cells.

RNN is also flexible in their inputs and outputs, for both sequence and single vector values.

We can actually, create entire layers of Recurrent Neurons.

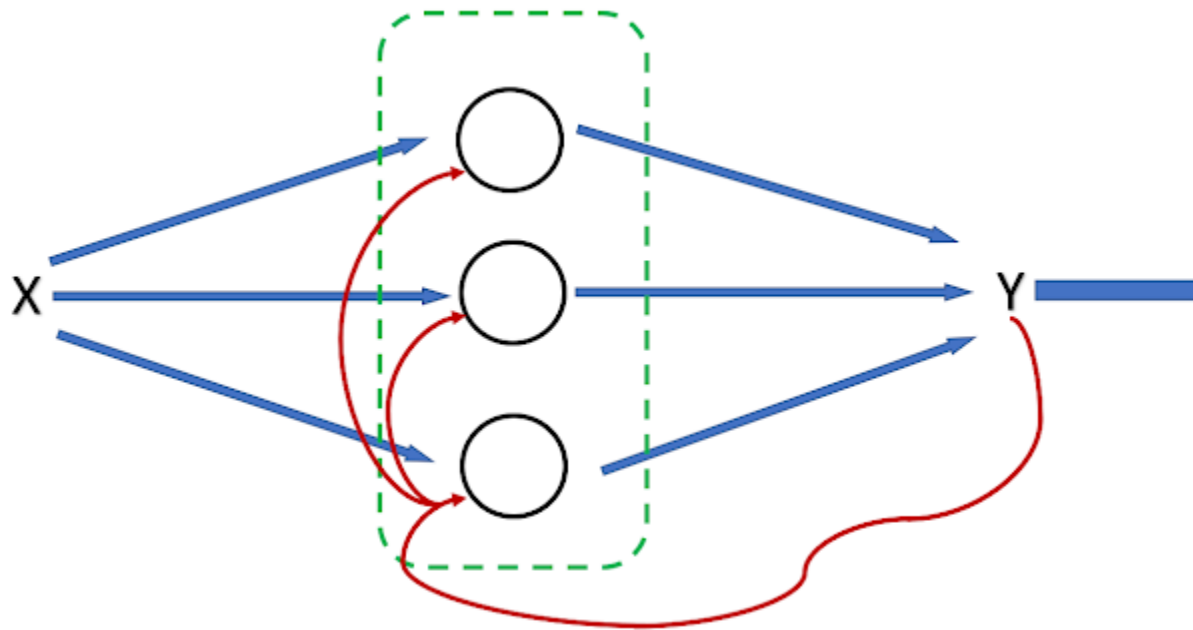
Here is an simple diagrammatic representation of it.

(Artificial Neural Network) ANN :



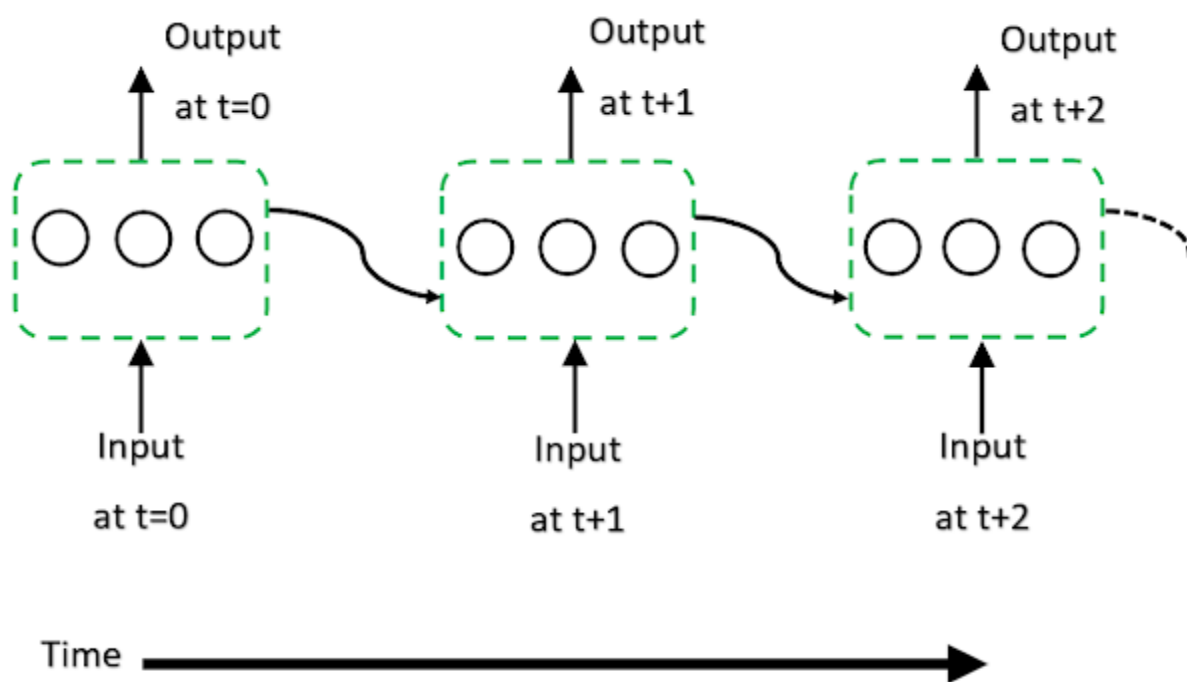
NORMAL ANN

(Recurrent Neural Network) RNN :



NORMAL RNN

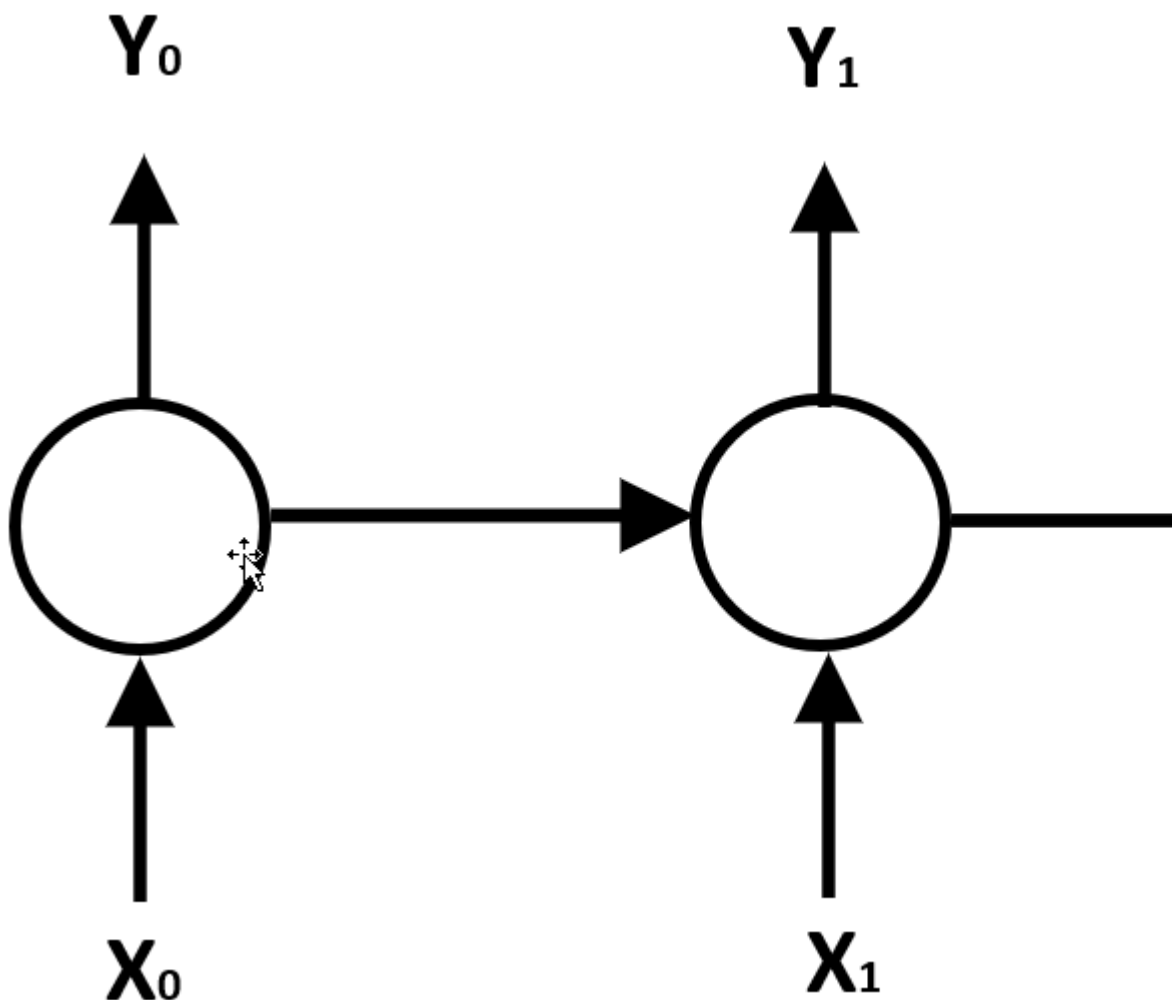
Now we can just unroll this layer in the same fashion. We pass input as time $t = 0$ into the layer and then the output of the layer is time $t+1$, than $t+2$, and so on.



RNN UNROLLED LAYER

As I have mentioned earlier that the RNN are very flexible in their inputs and outputs. There are different types of architectures we can use here. Let's see some of them:

1.) Sequence to Sequence (Many to Many):

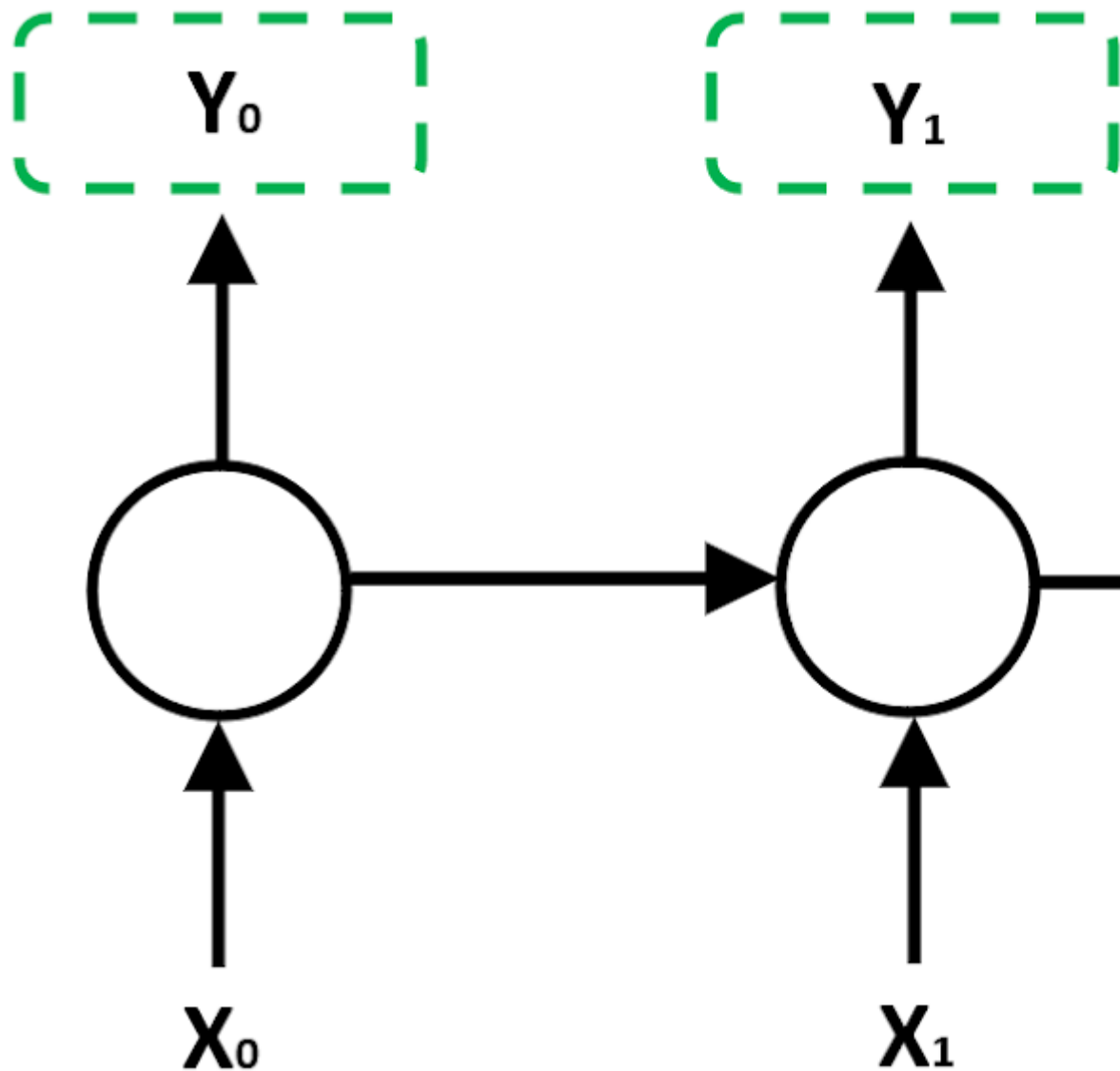


SEQUENCE TO SEQUENCE

In this, you pass in a sequence and you expect a sequence out.

For Example : We have an input of 5 words now you have to predict the output of 5 words.

2.) Sequence to Vector (Many to One):

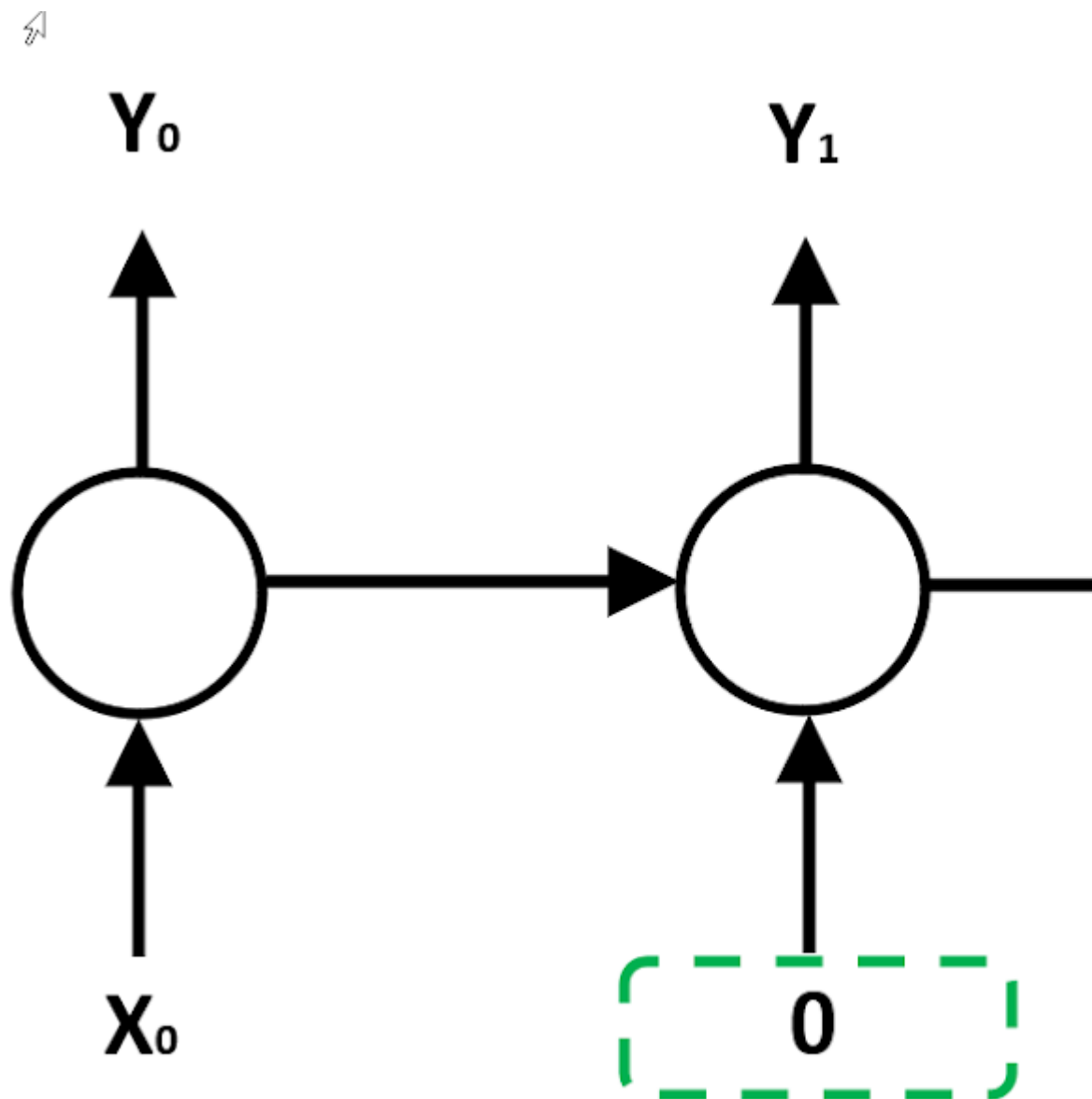


SEQUENCE TO VECTOR

In this, you pass in a sequence and you expect a vector as an out.

For Example : We have an input of 5 words now you have to predict the output of the next 1 words.

3.) Vector to Sequence (One to Many):



VECTOR TO SEQUENCE

In this you pass in a vector i.e a single value and you expect a sequence as an out.

For Example : We have the input of 1 word now you have to predict the output of the next 5 words.

A basic RNN has a major disadvantage, we only really “remember” the previous output. If we think back that unrolled diagram we were only feeding in the of 1 timestamp into the past and what happens is if we have really long histories we begin to start to forget the older historical samples since we are only really looking at the out of the last previous $t-1$ and it will be really great if we could keep the track of long history and not just that short term memory. Another issue that arises during training is the “**vanishing gradient**”.

Before directly diving into the LSTM (Long Short Term Memory Units). We will have an overview of Vanishing Gradient.

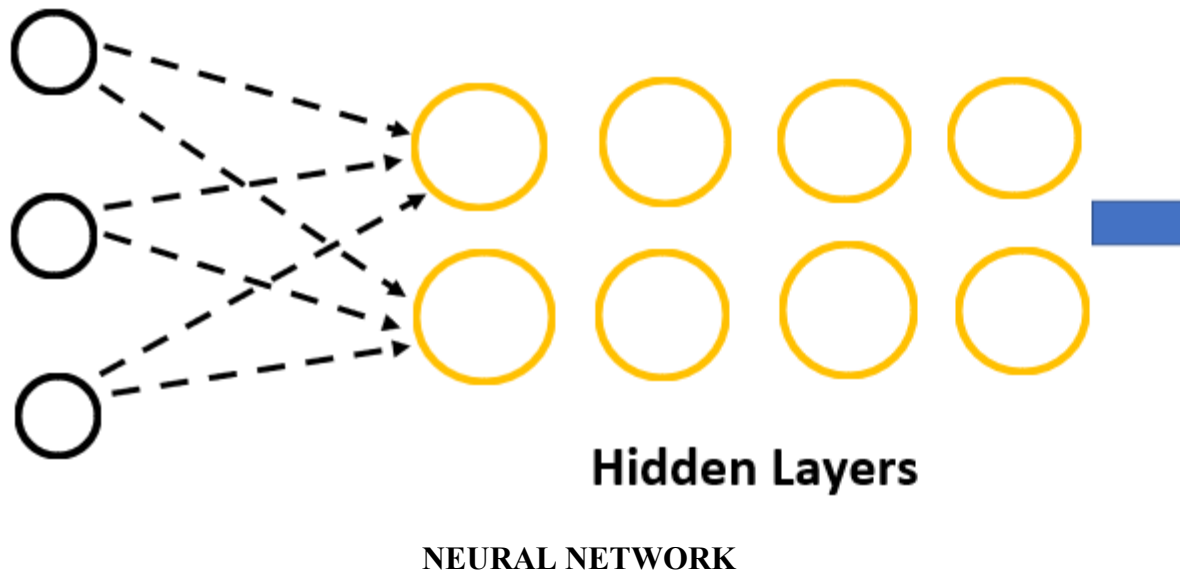
Let’s discuss the 2 main issues i.e **Exploding & Vanishing** Gradients.

As our networks grow deeper and more complex, we have 2 issues arises:

- Exploding Gradients**
- Vanishing Gradients**

Recall that the gradient is used in our calculation to adjust weights and biases in our network, if you don’t have any idea about Gradient you may check our [*GradientDescent*](#) article.

This errors might arise during backpropagation :

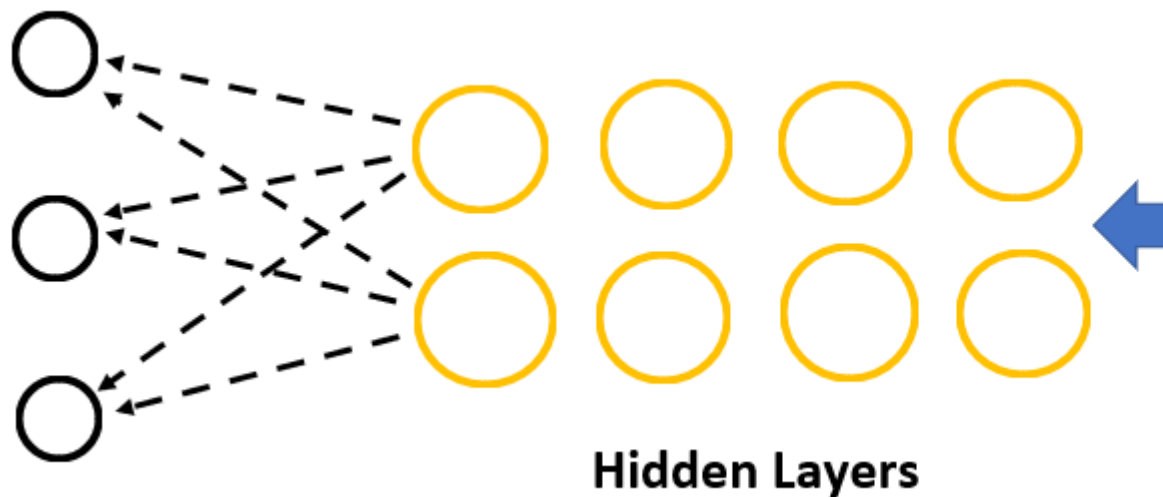


Now for complex data we such as complex image data or complex sequence data we end up needing deeper networks i.e we need more hidden layers in order to actually learn the patterns that are in our data.

Now what happens is there's this vanishing or exploding gradient issue that arises during the backpropagation step. So recall we are going to calculate some sort of loss metrics on the output layer and then backpropagation error all the way back to the input layer and if we have a lot's of hidden layers then we're having the update to the weights and biases be a function of many other derivatives that we're calculating along the way back.

- So backpropagation goes backward from the output to the input layer, propagating the error gradient.

- For deeper networks issues can arrive from backpropagation, vanishing and exploding gradients.

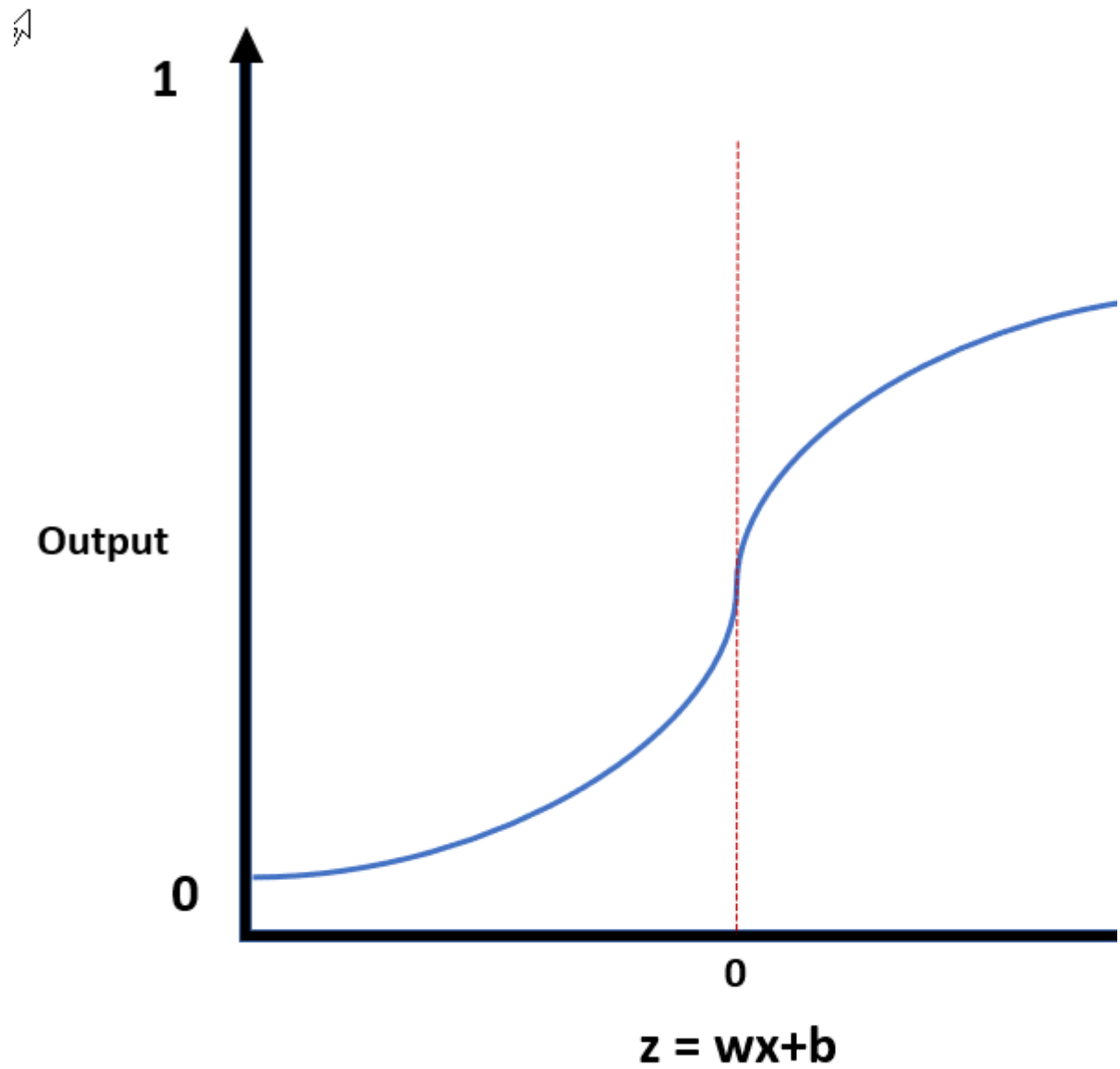


BACK PROPAGATION

- As you go back to the “lower” layers closer to the input layer, gradients often get smaller, to a vanishing gradient is usually the more common problem although they can technically explode on the way back. But as you are going back and back closer to those input layers there are gradients are getting smaller and eventually what happens is they’re so small by the time it gets to the input layer that the weights never really change that much at those lower input levels.
- That’s actually, a big problem because we want to be able to detect larger basic patterns in our data right out close to the input layer and have the deeper layers focus on the smaller details or in smaller details and patterns.

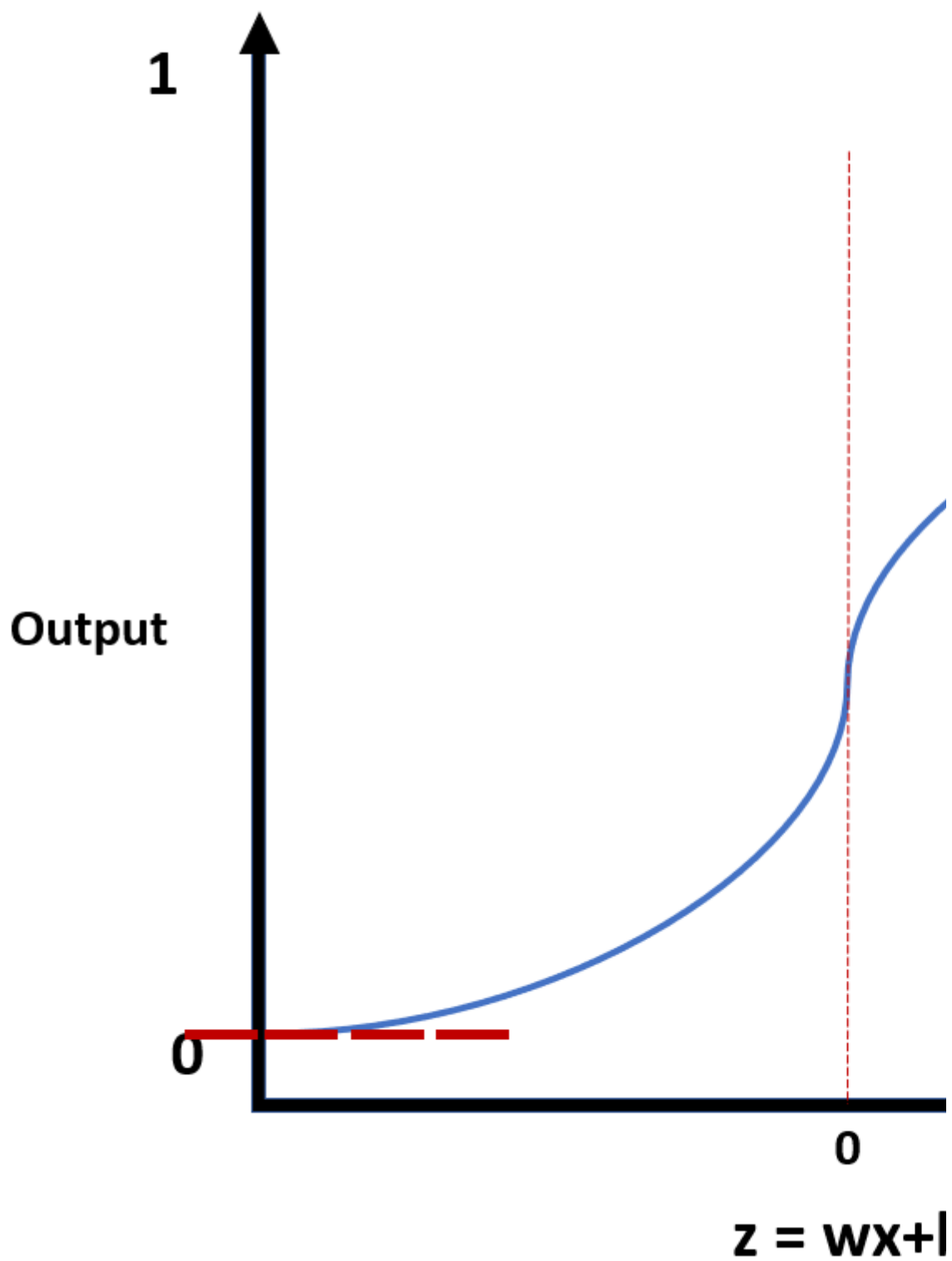
- And yes the opposite can also occur that the gradients explode on the way back, causing issues.

Now let's discuss why this is actually occurring and how we can fix it and let's also discuss how these issues specifically affect the RNN and how we can use the LSTM units and get a recurrent unit to also fix this to understand what actually happening let's take a look at a really common activation function such as sigmoid. Now as we know that the sigmoid activation function squeezes the input to fit between 0 & 1.



SIGMOID

However let's take a closer look at what happens when your inputs start to get further away from zero.



SIGMOID

The further away your input is from zero. The rate of change in the sigmoid function is actually decreasing rapidly and that

rate of change that is the derivatives of the sigmoid functions and we already know that the backpropagation and the gradient calculation is essentially just calculating that derivative in multiple dimensions as you go back through into the hidden layers.

** When N hidden layers use an activation like the sigmoid function, N small derivatives are multiplied together.*

** The gradient could decrease exponentially as we propagate down to the initial layers.*

** We can use other activation function such as ReLU which doesn't actually saturate those larger positive values.*

** The main benefit of using ReLU here is that doesn't matter how large your input value is going to be beyond 0. You are not going to exponentially decrease the rate of change.*

The other a possible solution is the **Batch Normalization** where your model will normalize each batch using that particular batches mean and standard deviation, and that has also been founded to alleviate the issue of vanishing gradient descent.

So apart from things such as **Batch Normalization**, researchers have also used “Gradient Clipping”, where gradients are cut off before reaching a

predetermined limit (eg: Cutt off gradients to be between -1 and 1).

RNN for Time Series presents their own Gradient Challenges, let's explore special **LSTM (Long Short Term Memory)** neuron units that help fix these issues!

*LSTM (Long Short Term Memory):

Many of the solutions previously presented for the vanishing gradients can also apply to RNN: different activation functions (ReLU), batch normalizations, etc...

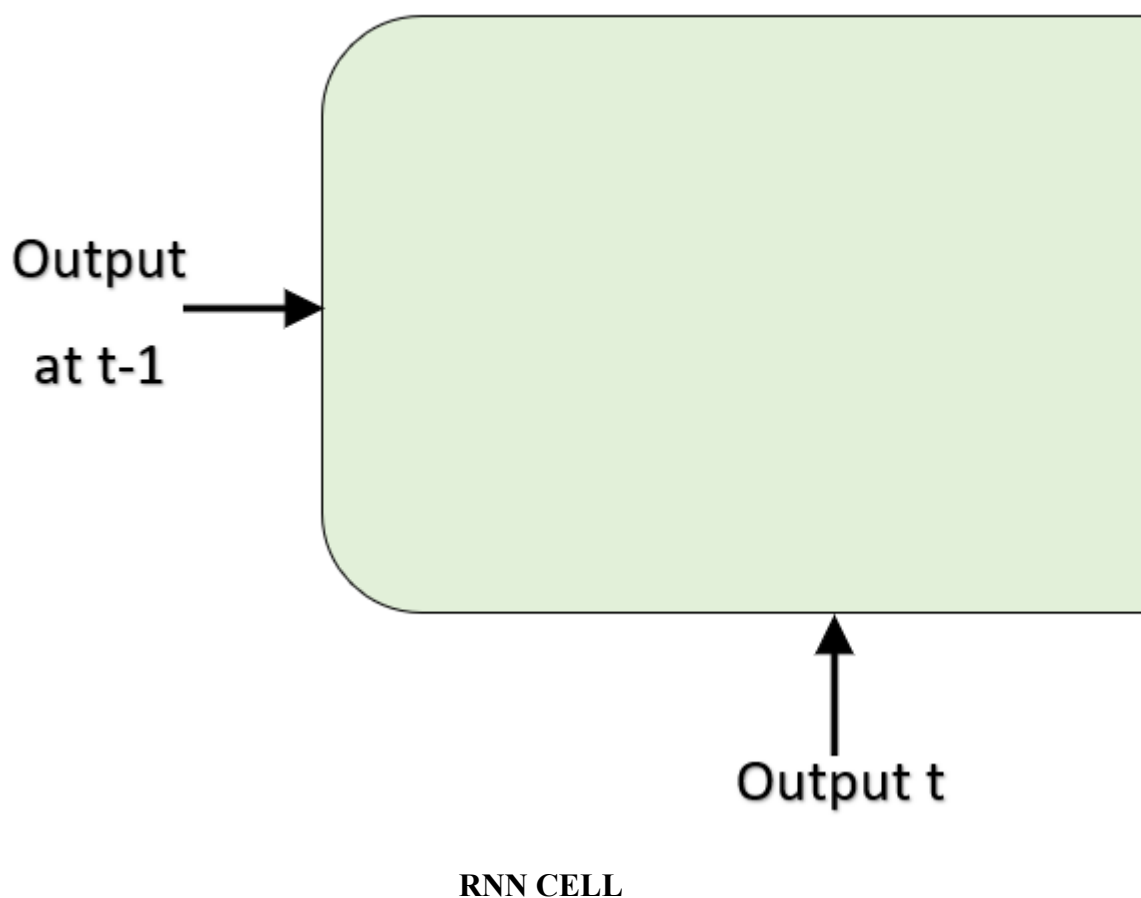
However because of the length of time series input, these could slow down the training.

A possible solution would be to just shorten the time steps used to prediction, but this makes the model worse at predicting longer trends. So maybe looking back 20 time steps for the 21st prediction you just look back 5-time steps to get the next prediction. However, this makes the model worse at predicting longer trends. So we still want to be able to use a long time sequence in order to predict the next item in the sequence.

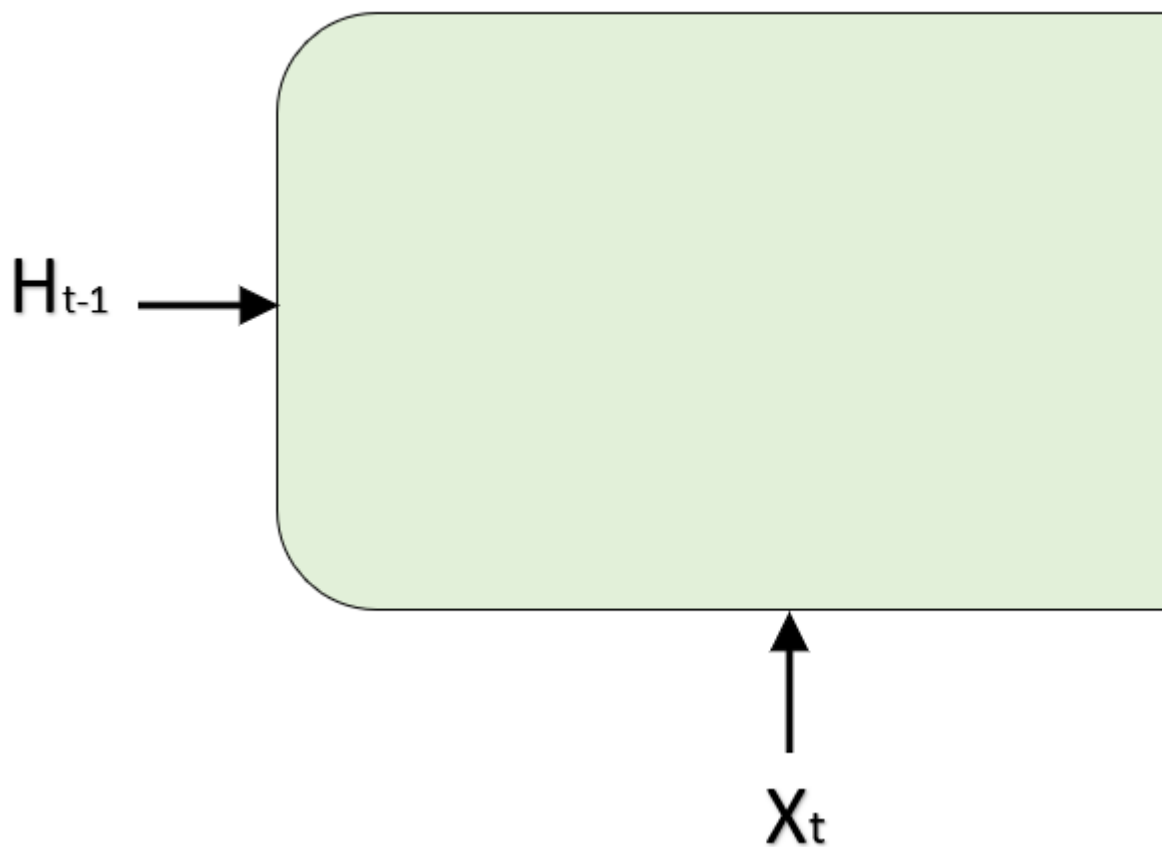
Another issue RNN faces are that after awhile the network will begin to “forget” the first inputs, as information is lost at each step going through the RNN.

We need some sort of “long-term memory” for our networks. So this is where LSTM (Long Short Term Memory) cell was created to help address these RNN issues.

Let's see the working of an LSTM cell:

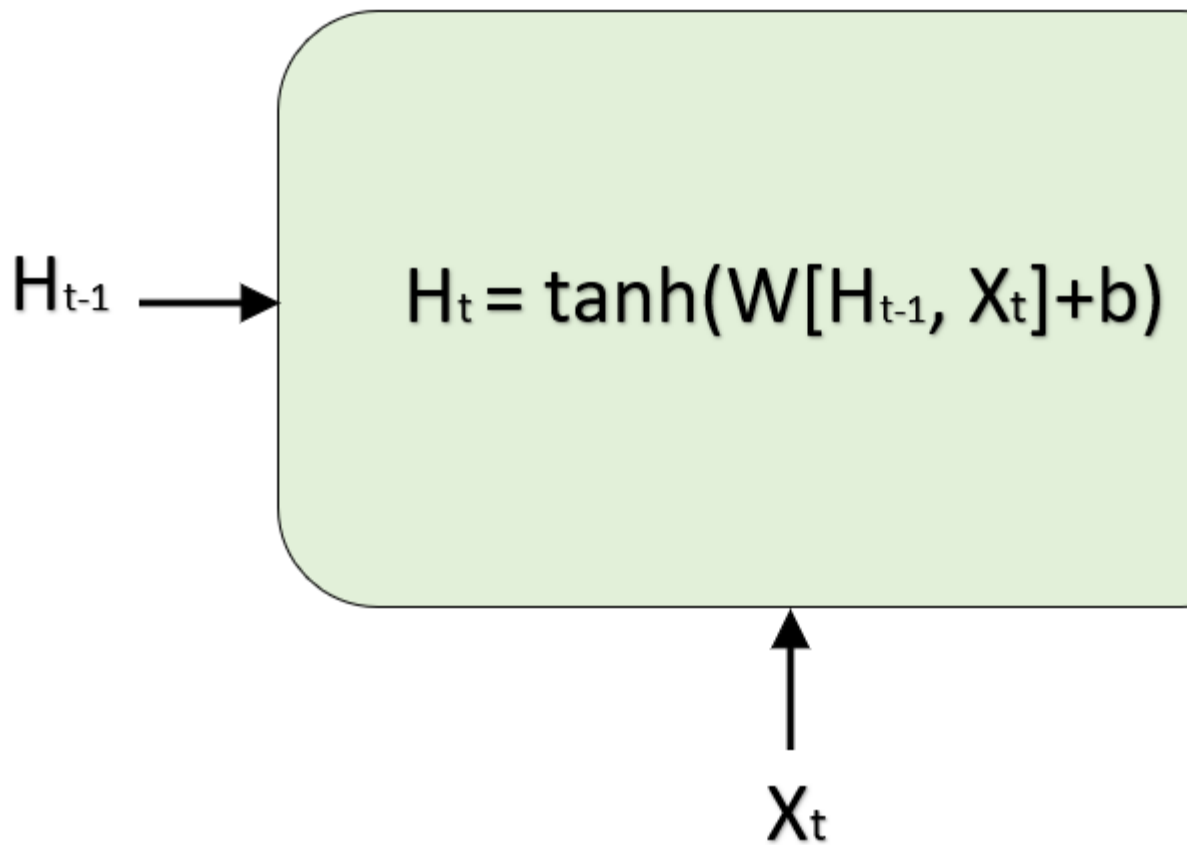


Take a look at what a single Recurrent neuron would actually be doing is essentially it's taking in both the previous output and then the current input and then producing the next output.



RNN CELL LABELED

instead of saying it output or input, we will refer to these as hidden state and then our current feature X going in. So basically we have H_{t-1} going along with X of T and that produces H_t . So in a standard RNN essentially what we do is we just have a single hyperbolic tangent function and then what we are doing, we are combining H_{t-1} with H_t multiply that with some **weight** matrix then adding a **bias** to it and then passing it through the hyperbolic tangent function and that gives us back our H_t .



RNN CELL LABELED FORMULA

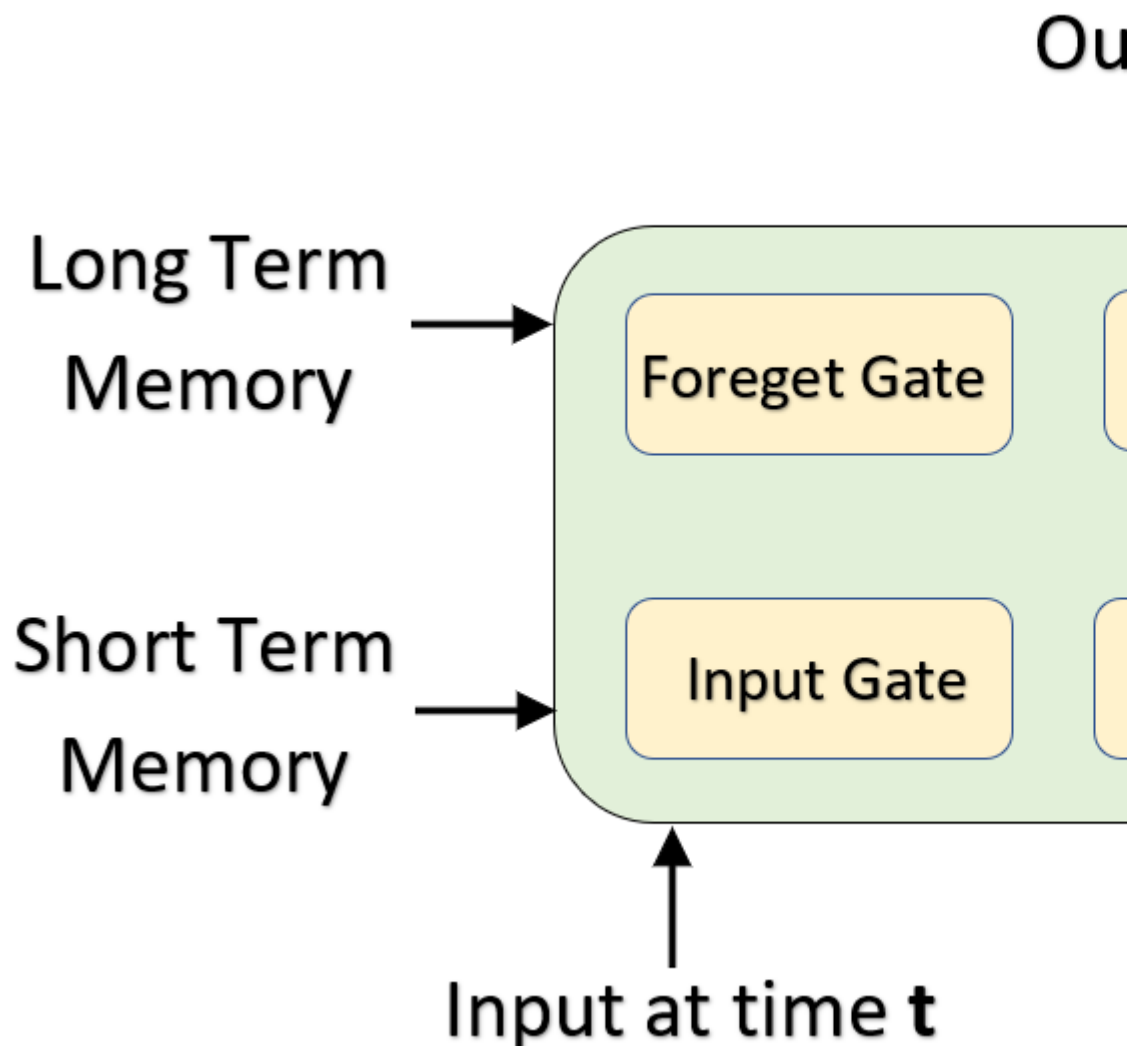
And we just repeat that through the next recurrent neuron or an extra current layer.

So LSTM also have this chain-like structure. But a repeating module have slight difference to it and instead of just having a single NN layer. There would be actually going to be 4 layers working and interacting in a special way and the way we end up getting 4 is the fact that not only will we keep track of just a single historical memory with H_{t-1} we're keeping track of both long term memory input and the short term memory input and that creating a new long term memory output and then use short term memory output along with the current input at time t and then we produce the output with time t .

STEPS OF LSTM :

Before diving into the steps let's see some of the notation we're going to be using so essentially.

So essentially we are going to have 4 main components inside the LSTM.



LSTM 4 MAIN COMPONENTS

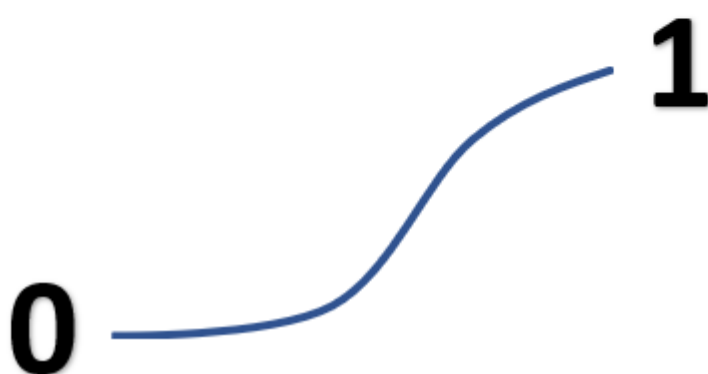
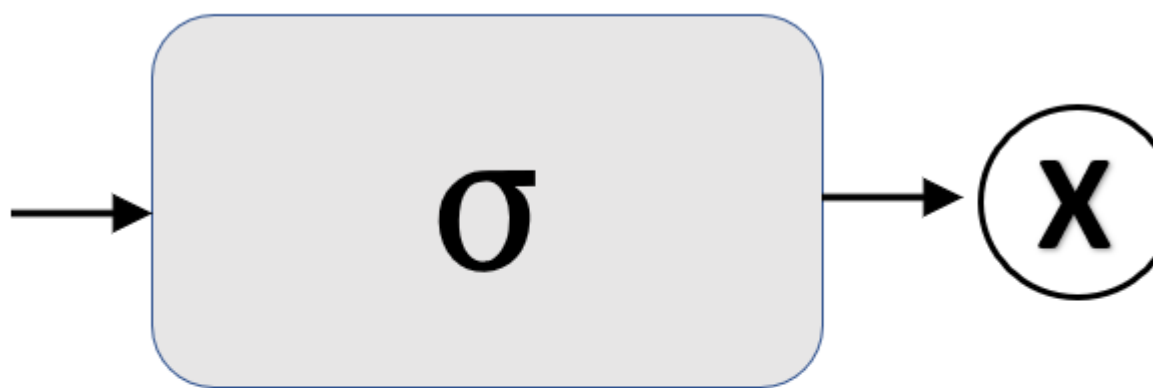
We have: **Forget Gate**, **Output Gate**, **Update Gate**, and an **Input Gate**.

***Forget Gate**: As the name suggests, Forget Gate will decide what to forget from the previous memory units.

***Input Gate**: As the name suggests, Input Gate will decide what actually accept into the neuron.

***Update Gate**: As the name suggests, Update Gate will update the memories.

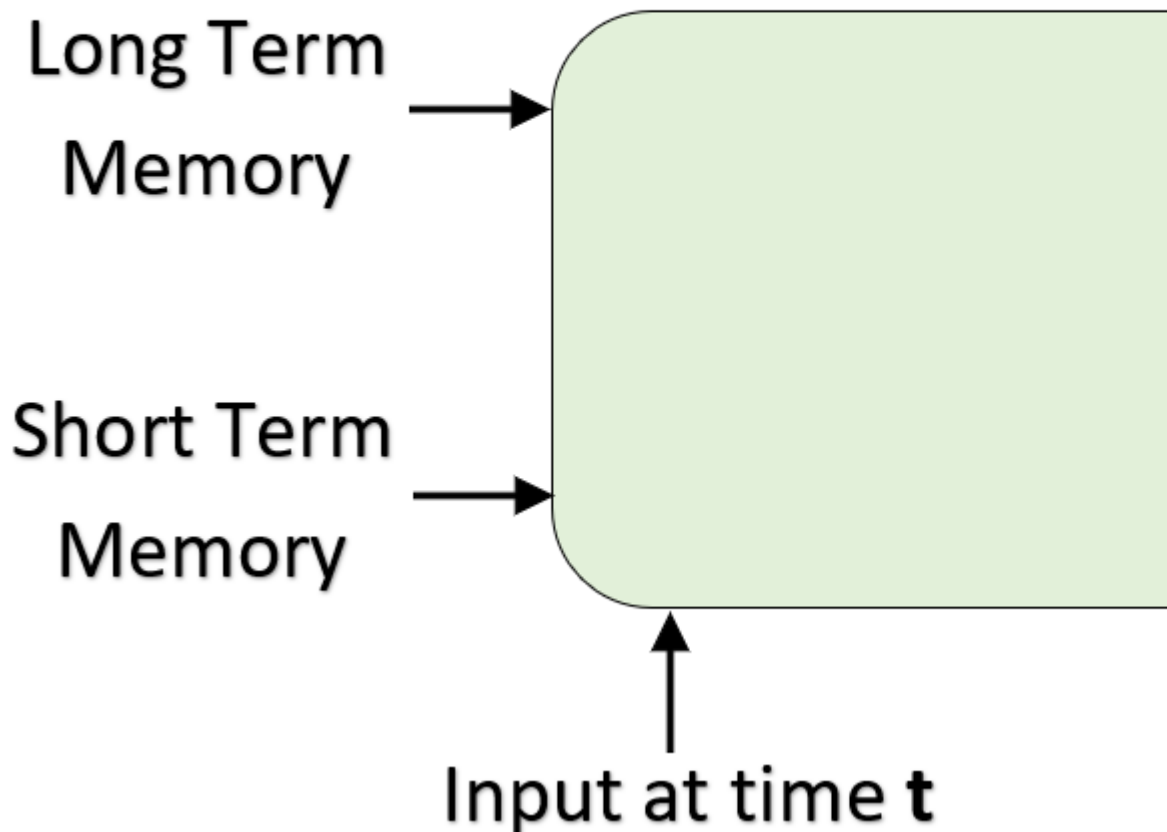
***Output Gate**: As the the the name suggests, Output Gate will actually output the new long term memory.



SIGMOID CELL

A Gate optionally lets some information through and essentially we can just think of this as mathematically it's a sigmoid function. It's either going to end up a sequence between a 0 or 1 and if it's 0 we don't let that information go and if it's a 1 we let it go.

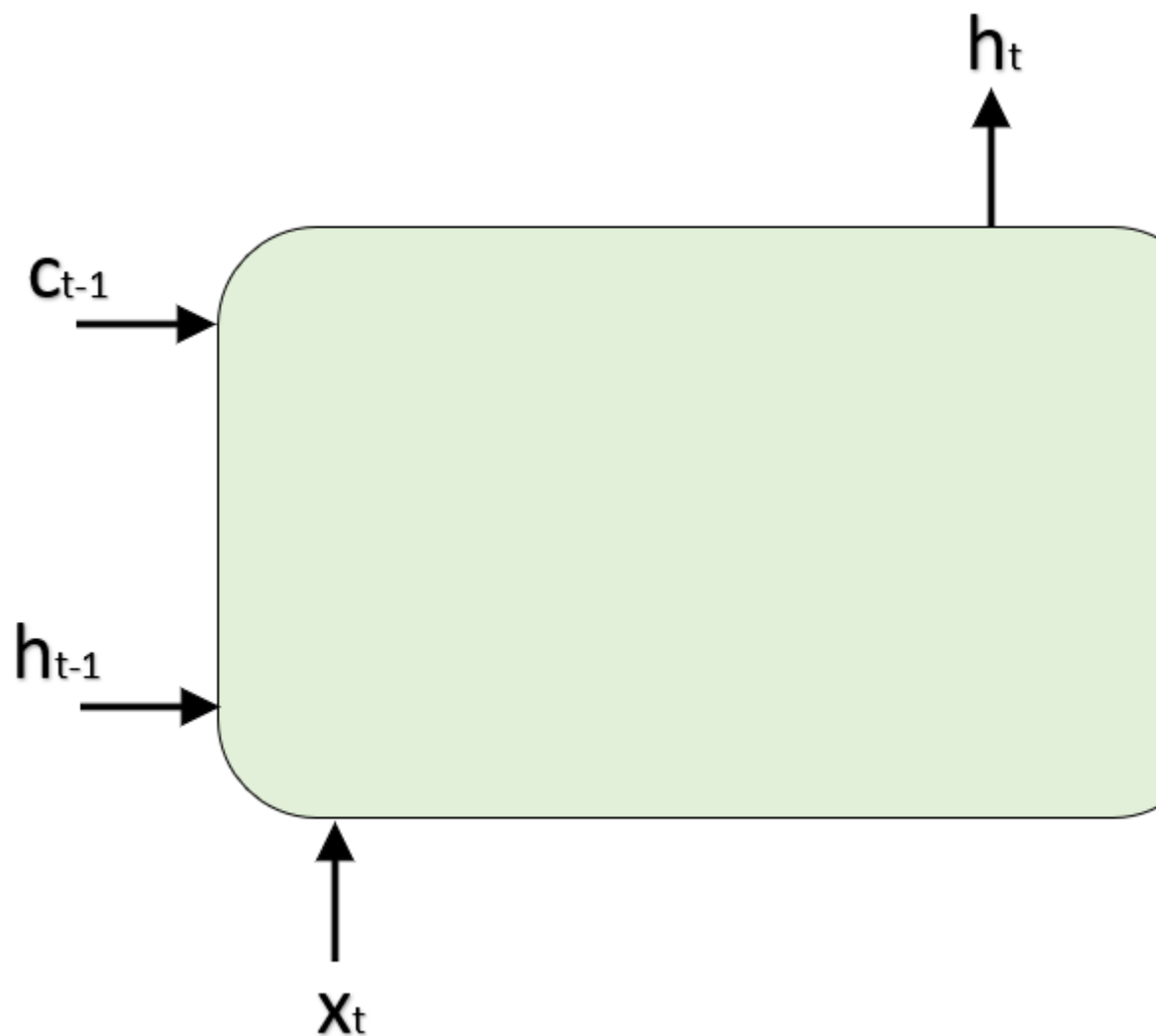
Out|



GENERAL STRUCTURE OF LSTM

So here is the general structure of LSTM and we are accepting both **Long Term** and **Short Term** memory, we can think of this as going to be passed in through conveyor belts inside of this neuron and what we end up happening are it just ends up kind of running down straight the entire chain and has some kind of linear interactions with a few functions inside of the cell.

Now for the purpose of a mathematical notation we're gonna relabel some of these and we're going to label them as such.

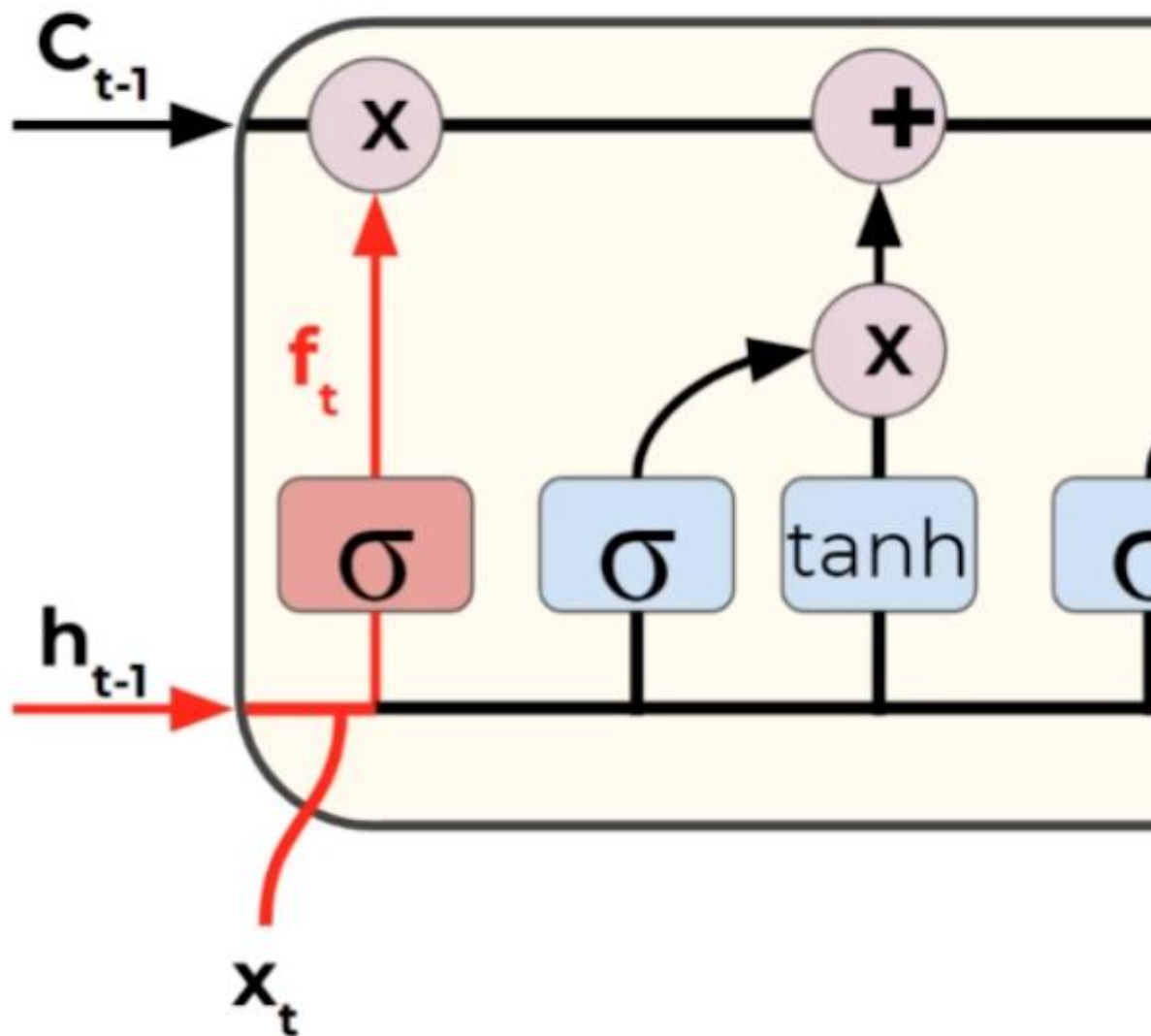


GENERAL STRUCTURE WITH NOTATION

So what are the actual linear interactions and functions going on inside of LSTM? Well, here we can see the entire LSTM cell.

Now let's go through the process step by step :

1.] The 1st step of LSTM is to decide what information is going to throw away from the cell state essentially what we are going to forget? So we end up creating is a forget gate layer or f_t .

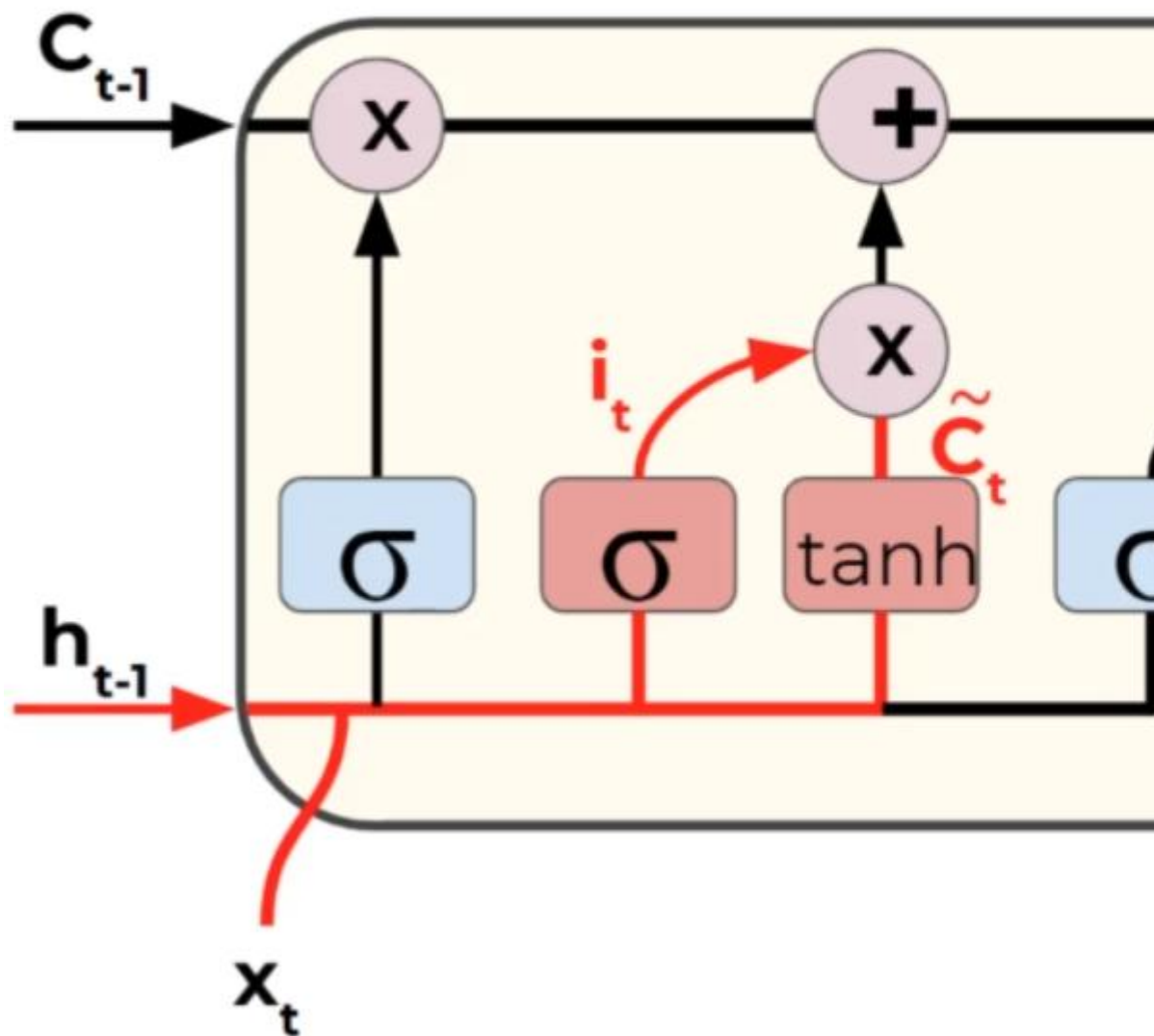


INSIDE OF LSTM

So we end up creating is a forget gate layer of f_t . Remember those gates are essentially just passing things through a sigmoid function where the closer it is to 0 that means fewer weights we are giving it. The closer it is to 1 the more weights we are giving it. So in the context of the forget gate layer the closer it is to 0 means forget about it and get rid of it and if it

closer to 1 than remember this it's important. So this is what f_t is doing. Notice that it's essentially a linear combination of h_{t-1} that previously hidden state combines with the input X_t . Then we have our own sets of weights for this forget gate layer plus bias and then we pass it through an activation function and then we get f_t then the next step after this is to decide **what new information are we going to store into the cell state?**

2.] In the 2nd step we have to decide what new information we are going to store into the cell state. This has 2 parts to it :



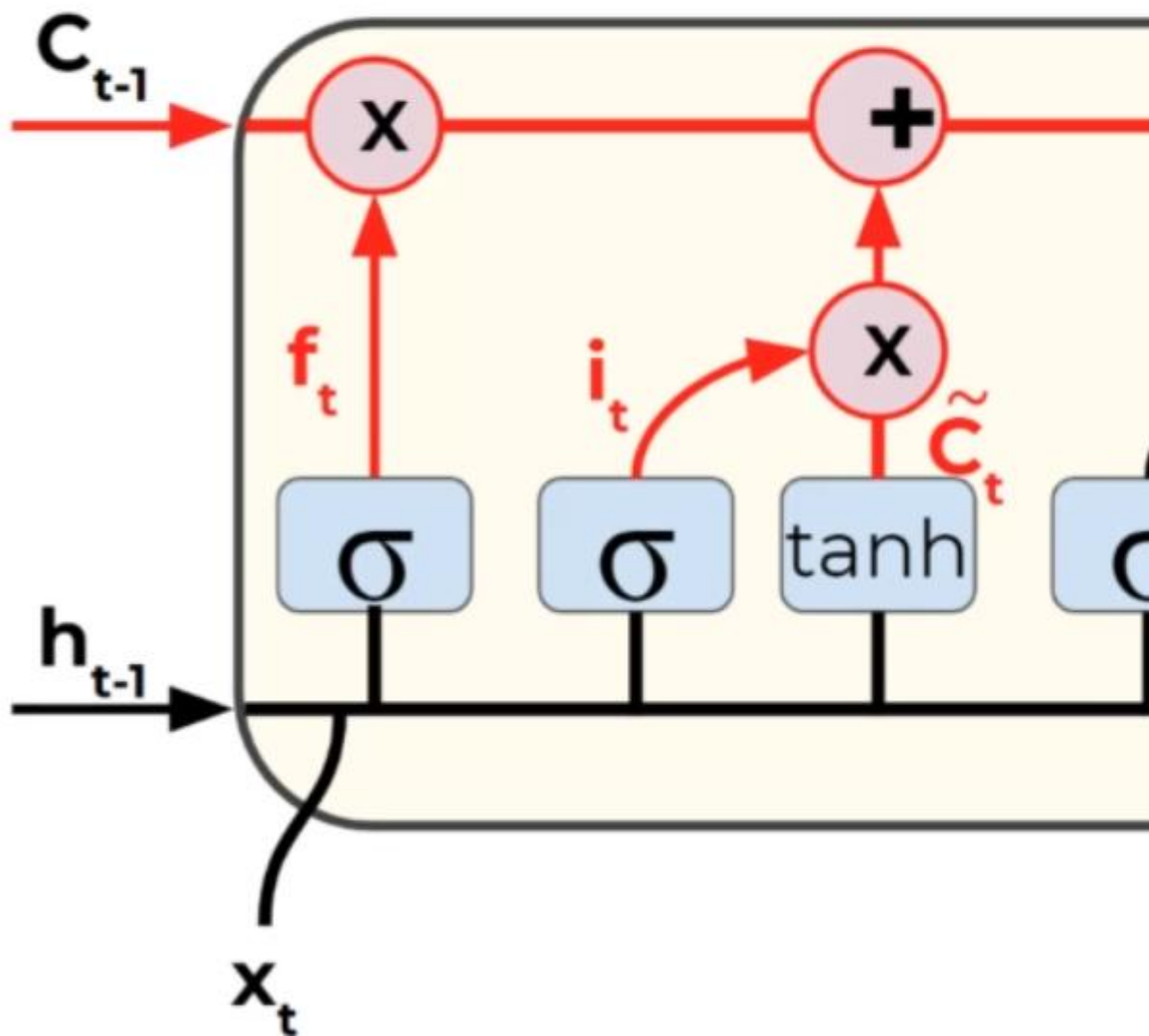
INSIDE OF LSTM STEP 2

(i) We have a sigmoid layer that we're going to label the input gate layer which is essentially going to decide what values are we going to update.

(ii) We have a hyperbolic tangent layer that creates a vector of new candidate values which will say C_t but we will label this

with a tilde on top of it. These are the new candidate values that will eventually be helpful in some sort of weighing before updating the cell state. So we have that input gate layer deciding which value we're going to update in the hyperbolic tangent layer which is creating a vector of those new candidate values.

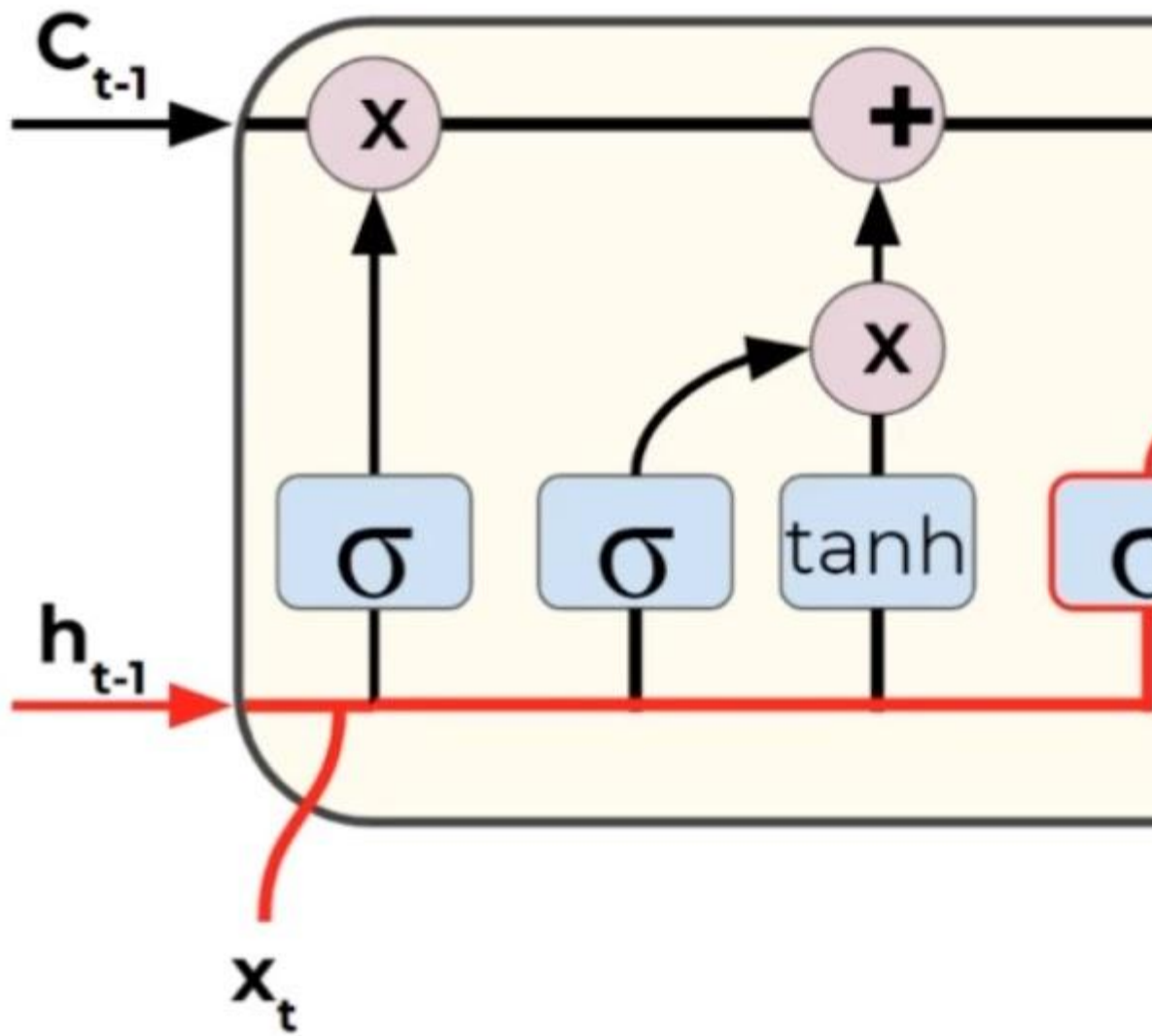
3.] Now it's time to update the old cell state which is C_{t-1} . In order to calculate the new cell state C_t that we're going to end up outputting.



INSIDE OF LSTM STEP 3

So what we are doing is that we multiply the old state by f_t forgetting the things that we decided weren't that important due to the forget gate layer. Then we add i_t times the new candidate values. So essentially these are the new candidate values for the cell state scaled by how much we decided to update each state value.

4.] Finally we have to decide what we are going to output? So this output will be based on our cell state. That actually a filtered version. First, we will end up doing is we run a sigmoid so that top equation which decides what parts of the cell state we are going to output. Then we put that cell state through the hyperbolic tangent function and what the hyperbolic tangent does is it pushes all the values to be between -1 and 1 and then we're gonna multiply it by the output of that initial sigmoid gate of O_t so that we only output the parts of that we decided to.



INSIDE OF LSTM STEP 4

Before directly diving into the coding first, get the data :

Google_Data

This data set contains:-

- 1.) **Data:** Stock information date.
- 2.) **Open:** Opening on a particular date.
- 3.) **High:** Highest price on that data.
- 4.) **Low:** Lowest price on that date.
- 5.) **Close:** Stock pricing closed that date.
- 6.) **Adj Close:** Adjusted close price.
- 7.) **Volume:** Volume of share.

We will be reading this data in chunks of 30 days. So we will be training our neural network on 30 days data we will be predicting the 31st day and then similarity we will again skipping the last data and then again we will be taking from 1 to 31 days and then we will be predicting at 32nd day.

Steps To Build Stock Prediction Model:

- i.) Importing & Preprocessing Data
- ii.) Building The RNN model

- iii.) Predicting Values
- iv.) Checking The Accuracy

i.) Importing & Preprocessing Data:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
data = pd.read_csv(r"E:\New folder\GOOG.csv", date_parser=True)
data.head()
```

Output:

	Date	Open	High	Low	Close	Adj Clc
0	2019-07-12	1143.989990	1147.339966	1138.780029	1144.900024	1144.900024
1	2019-07-15	1146.859985	1150.819946	1139.400024	1150.339966	1150.339966
2	2019-07-16	1146.000000	1158.579956	1145.000000	1153.579956	1153.579956
3	2019-07-17	1150.969971	1158.359985	1145.770020	1146.349976	1146.349976
4	2019-07-18	1141.739990	1147.604980	1132.729980	1146.329956	1146.329956

DATA

```
# Divide Data into Training and Test set
```

```
# Training Data
```

```
data_training = data[data['Date']<'2020-05-01'].copy()
data_training
```

Output:

	Date	Open	High	Low	Close	Adj
0	2019-07-12	1143.989990	1147.339966	1138.780029	1144.900024	1144.9
1	2019-07-15	1146.859985	1150.819946	1139.400024	1150.339966	1150.3
2	2019-07-16	1146.000000	1158.579956	1145.000000	1153.579956	1153.5
3	2019-07-17	1150.969971	1158.359985	1145.770020	1146.349976	1146.3
4	2019-07-18	1141.739990	1147.604980	1132.729980	1146.329956	1146.3
...	...	...	...	...	...	...
198	2020-04-24	1261.170044	1280.400024	1249.449951	1279.310059	1279.3
199	2020-04-27	1296.000000	1296.150024	1269.000000	1275.880005	1275.8
200	2020-04-28	1287.930054	1288.050049	1232.199951	1233.670044	1233.6
201	2020-04-29	1341.459961	1359.989990	1325.339966	1341.479980	1341.4
202	2020-04-30	1324.880005	1352.819946	1322.489990	1348.660034	1348.6

203 rows × 7 columns

TRAINING DATA

```
# Divide Data into Training and Test set
```

```
# Test Data
```

```
data_test = data[data['Date']>='2020-05-01'].copy()  
data_test
```

Output:

	Date	Open	High	Low	Close	Adj
203	2020-05-01	1328.500000	1352.069946	1311.000000	1320.609985	1320.6
204	2020-05-04	1308.229980	1327.660034	1299.000000	1326.800049	1326.8
205	2020-05-05	1337.920044	1373.939941	1337.459961	1351.109985	1351.1
206	2020-05-06	1361.689941	1371.119995	1347.290039	1347.300049	1347.3
207	2020-05-07	1365.939941	1377.599976	1355.270020	1372.560059	1372.5
208	2020-05-08	1383.130005	1398.760010	1375.479980	1388.369995	1388.3
209	2020-05-11	1378.280029	1416.530029	1377.151978	1403.260010	1403.2
210	2020-05-12	1407.119995	1415.000000	1374.770020	1375.739990	1375.7
211	2020-05-13	1377.050049	1385.482056	1328.400024	1349.329956	1349.3
212	2020-05-14	1335.020020	1357.420044	1323.910034	1356.130005	1356.1
213	2020-05-15	1350.000000	1374.479980	1339.000000	1373.189941	1373.1
214	2020-05-18	1361.750000	1392.324951	1354.250000	1383.939941	1383.9
215	2020-05-19	1386.996948	1392.000000	1373.484985	1373.484985	1373.4
216	2020-05-20	1389.579956	1410.420044	1387.250000	1406.719971	1406.7
217	2020-05-21	1408.000000	1415.489990	1393.449951	1402.800049	1402.8
218	2020-05-22	1396.709961	1412.760010	1391.829956	1410.420044	1410.4
219	2020-05-26	1437.270020	1441.000000	1412.130005	1417.020020	1417.0
220	2020-05-27	1417.250000	1421.739990	1391.290039	1417.839966	1417.8
221	2020-05-28	1396.859985	1440.839966	1396.000000	1416.729980	1416.7
222	2020-05-29	1416.939941	1432.569946	1413.349976	1428.920044	1428.9
223	2020-06-01	1418.390015	1437.959961	1418.000000	1431.819946	1431.8
224	2020-06-02	1430.550049	1439.609985	1418.829956	1439.219971	1439.2
225	2020-06-03	1438.300049	1446.552002	1429.776978	1436.380005	1436.3
226	2020-06-04	1430.400024	1438.959961	1404.729980	1412.180054	1412.1
227	2020-06-05	1413.170044	1445.050049	1406.000000	1438.390015	1438.3
228	2020-06-08	1422.339966	1447.989990	1422.339966	1446.609985	1446.6
229	2020-06-09	1445.359985	1468.000000	1443.209961	1456.160034	1456.1
230	2020-06-10	1459.540039	1474.259033	1456.270020	1465.849976	1465.8
231	2020-06-11	1442.479980	1454.474976	1402.000000	1403.839966	1403.8
232	2020-06-12	1428.489990	1437.000000	1386.020020	1413.180054	1413.1
233	2020-06-15	1390.800049	1424.800049	1387.920044	1419.849976	1419.8
234	2020-06-16	1445.219971	1455.020020	1425.900024	1442.719971	1442.7

TEST DATA

```
training_data = data_training.drop(['Date','Adj Close'],axis=1)
training_data.head()
```

Output:

	Open	High	Low	Close	Volume
0	1143.989990	1147.339966	1138.780029	1144.900024	864000
1	1146.859985	1150.819946	1139.400024	1150.339966	903800
2	1146.000000	1158.579956	1145.000000	1153.579956	1238800
3	1150.969971	1158.359985	1145.770020	1146.349976	1170000
4	1141.739990	1147.604980	1132.729980	1146.329956	1291300

TRAINING DATA HEAD

Here we have to predict the **Open** i.e opening price of that particular stock.

```
# Data Preprocessing
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler()
```

```
training_data = scaler.fit_transform(training_data)
training_data
```

Output:

```
array([[0.18669966, 0.16497901, 0.24660936, 0.1878019 , 0.1158
        [0.1928248 , 0.17253128, 0.24783015, 0.19937452, 0.1247
        [0.19098942, 0.18937209, 0.25885667, 0.20626709, 0.1999
        ...,
        [0.49389635, 0.4703487 , 0.43055608, 0.37664618, 0.5846
        [0.60813981, 0.62647311, 0.61395166, 0.60599489, 0.7729
        [0.57275489, 0.61091264, 0.60833997, 0.62126933, 0.5199
```

MIN-MAX SCALER DATA

```
X_train = []
y_train = []

for i in range(30, training_data.shape[0]):
    X_train.append(training_data[i-30:i])
    y_train.append(training_data[i,0])

X_train, y_train = np.array(X_train), np.array(y_train)

X_train.shape
```

Output:

(173, 30, 5)

ii.) Build The RNN Model:

```
# Building LSTM

from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, LSTM, Dropout

regression = Sequential()
regression.add(LSTM(units=50, activation="relu", return_sequences=True,
input_shape=(X_train.shape[1], 5)))
regression.add(Dropout(0.2))
```

```
regression.add(LSTM(units=50, activation="relu", return_sequences=True,  
input_shape=(X_train.shape[1], 5)))  
regression.add(Dropout(0.3))  
  
regression.add(LSTM(units=50, activation="relu", return_sequences=True,  
input_shape=(X_train.shape[1], 5)))  
regression.add(Dropout(0.4))  
  
regression.add(LSTM(units=50, activation="relu"))  
regression.add(Dropout(0.5))  
  
regression.add(Dense(units=1))  
  
regression.summary()
```

Output:

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 30, 50)	11200
dropout (Dropout)	(None, 30, 50)	0
lstm_1 (LSTM)	(None, 30, 50)	20200
dropout_1 (Dropout)	(None, 30, 50)	0
lstm_2 (LSTM)	(None, 30, 50)	20200
dropout_2 (Dropout)	(None, 30, 50)	0
lstm_3 (LSTM)	(None, 50)	20200
dropout_3 (Dropout)	(None, 50)	0
dense (Dense)	(None, 1)	51
Total params: 71,851		
Trainable params: 71,851		
Non-trainable params: 0		

MODEL SUMMARY

Compile and Fit the model

```
regression.compile(optimizer='adam', loss="mean_squared_error")
regression.fit(X_train,y_train,epochs=550,batch_size=32)
```

Output:

```
Epoch 542/550
173/173 [=====] - 0s 2m
Epoch 543/550
173/173 [=====] - 0s 2m
Epoch 544/550
173/173 [=====] - 0s 2m
Epoch 545/550
173/173 [=====] - 0s 2m
Epoch 546/550
173/173 [=====] - 0s 2m
Epoch 547/550
173/173 [=====] - 0s 2m
Epoch 548/550
173/173 [=====] - 0s 2m
Epoch 549/550
173/173 [=====] - 0s 2m
Epoch 550/550
173/173 [=====] - 0s 2m

Out[14]: <tensorflow.python.keras.callbacks.History at 0x
```

COMPILE AND FIT DATA

iii.) Predict Values:

```
# Prepare test data
```

```
data_test.head()
```

Output:

	Date	Open	High	Low	Close	Adj
203	2020-05-01	1328.500000	1352.069946	1311.000000	1320.609985	1320.6
204	2020-05-04	1308.229980	1327.660034	1299.000000	1326.800049	1326.8
205	2020-05-05	1337.920044	1373.939941	1337.459961	1351.109985	1351.1
206	2020-05-06	1361.689941	1371.119995	1347.290039	1347.300049	1347.3
207	2020-05-07	1365.939941	1377.599976	1355.270020	1372.560059	1372.5

TEST HEAD

```
past_30_days = data_training.tail(30)

df = past_30_days.append(data_test, ignore_index=True)
df = df.drop(['Date', 'Adj Close'],axis=1)
df.head()
```

Output:

	Open	High	Low	Close	Volume
0	1093.050049	1157.969971	1060.108032	1115.290039	3651100
1	1135.719971	1143.989990	1065.489990	1072.319946	3601800
2	1061.319946	1071.319946	1013.536011	1056.619995	4044100
3	1103.770020	1135.000000	1090.619995	1134.459961	3344500
4	1126.469971	1148.900024	1086.010010	1102.489990	4081500

FILTERED DATA

```
inputs = scaler.transform(df)
inputs
```

Output:

```
array([[0.0779837 , 0.1880483 , 0.09170175, 0.12481131, 0.7409
[0.16904979, 0.15770888, 0.102299 , 0.03339918, 0.7299
[0.01026536, 0. , 0. , 0. , 0.8291
[0.10086225, 0.13819876, 0.15178076, 0.16559231, 0.6722
[0.14930846, 0.16836466, 0.14270355, 0.09758121, 0.8375
[0.11799993, 0.21409076, 0.1575107 , 0.22364758, 0.7231
[0.14760125, 0.17220596, 0.18188732, 0.11506791, 0.6417
[0.1462567 , 0.17428927, 0.16331925, 0.19188623, 0.4994
[0.19376398, 0.22567984, 0.24534915, 0.22590269, 0.4797
[0.13976865, 0.12667484, 0.16522915, 0.10423981, 0.4478
[0.0891028 , 0.12053326, 0.16316181, 0.1366179 , 0.3627
[0.1333981 , 0.1133283 , 0.13049566, 0.08777419, 0.4409
[0.17391583, 0.26767325, 0.23117198, 0.27719289, 0.5197
[0.35105432, 0.33351719, 0.33216366, 0.27632061, 0.4575
[0.32010844, 0.32064776, 0.34384012, 0.32688759, 0.3656
[0.35762756, 0.33475409, 0.36072446, 0.32937642, 0.4099
[0.32582821, 0.32377299, 0.3427335 , 0.34237472, 0.3122
[0.40246709, 0.45737066, 0.4398698 , 0.45229436, 0.4761
[0.40357692, 0.45387666, 0.44670228, 0.4379135 , 0.2976
[0.46438022, 0.45070824, 0.45107347, 0.44004084, 0.4868
[0.48732286, 0.48419462, 0.50740742, 0.48211975, 0.3592
[0.45776425, 0.45635072, 0.4879928 , 0.44672073, 0.3023
[0.40654349, 0.39703912, 0.38627259, 0.33977916, 0.4049
[0.40342764, 0.46505989, 0.44985268, 0.43948771, 0.3915
[0.45893817, 0.481764 , 0.49645973, 0.46735612, 0.2733
[0.43678518, 0.45374658, 0.46452187, 0.47373814, 0.2899
[0.51111922, 0.48792731, 0.50301652, 0.46644124, 0.2816
[0.49389635, 0.4703487 , 0.43055608, 0.37664618, 0.5846
[0.60813981, 0.62647311, 0.61395166, 0.60599489, 0.7729
[0.57275489, 0.61091264, 0.60833997, 0.62126933, 0.5199
[0.58048068, 0.60928499, 0.58571582, 0.56159725, 0.3869
[0.53722043, 0.55631048, 0.56208745, 0.57476564, 0.2594
[0.60058492, 0.65674736, 0.63781631, 0.62648121, 0.2924
[0.65131461, 0.6506275 , 0.65717204, 0.61837617, 0.1946
[0.66038495, 0.66469038, 0.67288487, 0.67211288, 0.2355
[0.69707196, 0.71061199, 0.71267891, 0.70574603, 0.2334
```

INPUT TRANSFORM DATA

```
X_test = []
y_test = []
```

```
for i in range(30, inputs.shape[0]):
```

```
X_test.append(inputs[i-30:i])
y_test.append(inputs[i,0])
```

```
X_test, y_test = np.array(X_test), np.array(y_test)
X_test.shape, y_test.shape
```

Output:

((49, 30, 5), (49,))

```
y_pred = regression.predict(X_test)
y_pred
```

Output:

```
[0.6231649 ],
[0.6231649 ],
[0.6606777 ],
[0.7072302 ],
[0.80955064],
[0.86017704],
[0.86500204],
[0.8601824 ],
[0.8548392 ],
[0.8259275 ],
[0.8476343 ],
[0.85563636],
[0.88016164],
[0.86826885],
[0.9088199 ],
[0.9138795 ],
[0.91115445],
[0.91142994],
[0.8429315 ],
[0.86644673],
[0.7974982 ],
[0.77378047],
[0.82571965],
[0.809147  ],
[0.8267869 ]], dtype=float32)
```

PREDICTED DATA

```
scaler.scale_
```

Output:

```
array([2.13419869e-03, 2.17020477e-03, 1.96903103e-03,  
2.12734298e-03, 2.24300742e-07])
```

```
scale = 1/2.13419869e-03  
scale
```

Output:

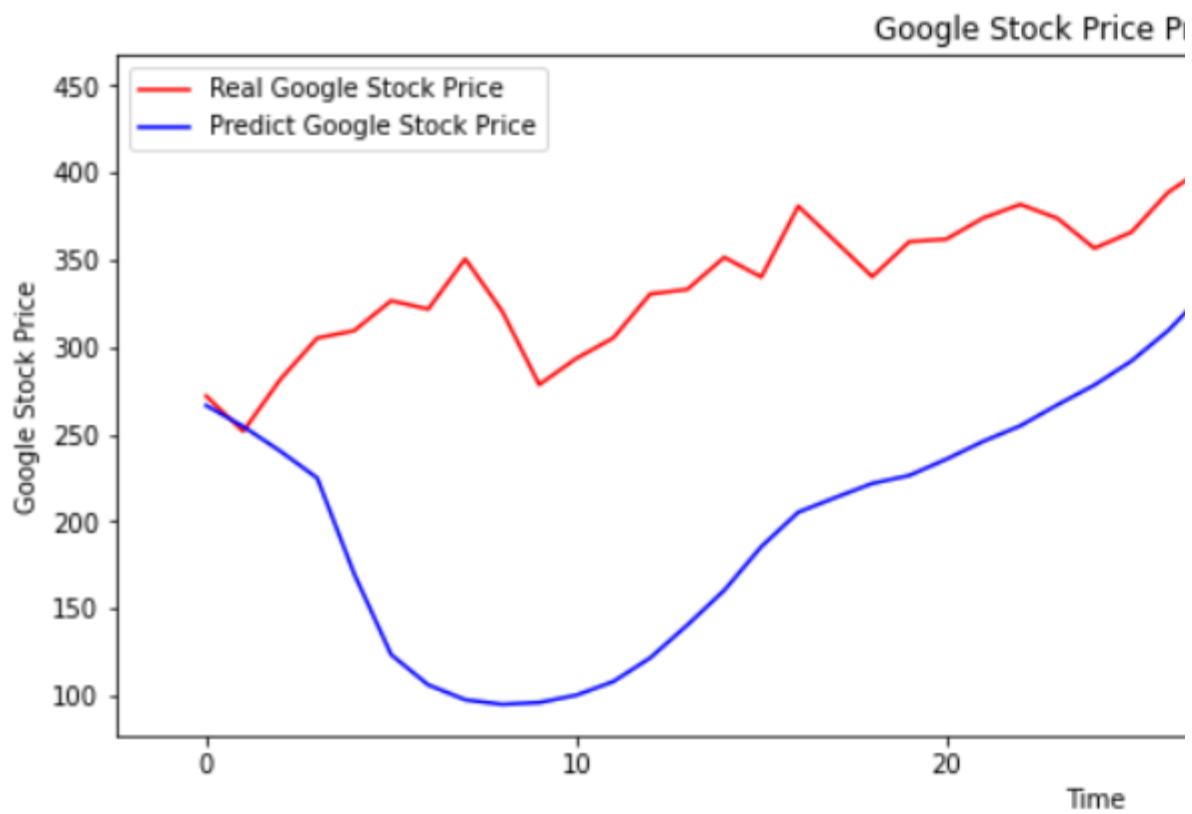
```
468.55993525138933
```

```
y_pred = y_pred * scale  
y_test = y_test * scale
```

vi.) Visualize The Data and Check Accuracy:

```
# Visualize the data  
  
plt.figure(figsize=(14,5))  
plt.plot(y_test, color="red", label="Real Google Stock Price")  
plt.plot(y_pred, color="blue", label="Predict Google Stock Price")  
plt.title("Google Stock Price Prediction")  
plt.xlabel('Time')  
plt.ylabel('Google Stock Price')  
plt.legend()  
plt.show()
```

Output:



DATA VISUALIZATION